Anomaly Detection in SCADA Systems: A State Transition Modeling

Nisha Kumari Barsha and Neminath Hubballi, *Senior Member, IEEE*

***Abstract*—Smart-Grid networks use Supervisory Control and Data Acquisition (SCADA) systems to bring measurement data from sensory nodes. These measurements drive the control decisions which are safety critical operations. SCADA communications now happen over TCP/IP networks and hence are susceptible to cyber attacks. As smart-grid is a critical infrastructure, it is essential to detect these cyber attacks. In this direction, our contributions in this paper are two-fold. First, we present three broad classes of network anomalies namely single message anomaly, message sequencing anomaly, and time based anomaly. We show that several cyber attacks in smart-grid networks can be detected by identifying these three types of anomalies. Second, we describe a novel state transition machine based model for identifying these three types of anomalies and hence different cyber attacks in smart-grid networks. Our state transition based model `Deterministic Counting Timed Automata (DCTA)` formalizes constraints on message attributes, timing of events, and counter values associated with states to detect these anomalies. We experiment with a publicly available dataset and show that `DCTA` is capable of detecting various cyber attacks with 100% detection rate in the best case for most of the attacks considered. We also benchmark its performance with recent methods found in the literature.**

***Index Terms*—Cyber attacks, smart-grid networks, SCADA, state transition model, anomaly detection.**

I. Introduction

INDUSTRIAL Control Systems (ICS) are implemented to monitor and control critical infrastructure like electric grids, water distribution networks, etc. Typically Supervisory Control and Data Acquisition (SCADA) systems are used for controlling and monitoring these infrastructures. SCADA systems bring measurement data to a control center for making control decisions. Power grid networks are one of the prominent applications that use SCADA for communication. There are many proprietary protocols [1], [2] designed for communicating these measurements to the control unit/master. Among these Modbus [3], and IEC 60870-5-104 protocol [4] gained popularity as their design was made public and standardised. However, these communication protocols lack adequate security as traditionally SCADA networks were isolated from the Internet. However, modern SCADA systems integrate with Internet using TCP/IP communication; and hence are susceptible to cyber attacks and any faults or malfunctions in these systems can have serious consequences [5], [6]. Cyber attack on the Ukrainian Grid network [7] is an example incident.

Traditional protection methods used for safeguarding this critical infrastructure appear in the form of installing firewalls and intrusion detection systems. However, these techniques are not tailored for protecting SCADA system communications. Further, these systems may have challenges in monitoring internal communications as these are usually installed at the perimeter of the network. A cyber attack can be initiated through a compromised device within the SCADA infrastructure through malware or other means. Thus, it is essential to monitor application layer communications analyzing the SCADA communications in order to detect anomalies/cyber attacks against these systems. Few earlier works [8], [9] have shown that monitoring flow level information of SCADA communications is an effective way to detect attacks. Owing to the periodic nature of communication in SCADA systems, several attack detection models [9], [10], [11], [12] described state transition machine based detection systems. Such state transition machines can identify the sequence of communications/messages and flag unusual patterns. However, these methods identify the unusual sequence of messages and may not help in detecting all variants of attacks. For e.g., a Denial of Service (DoS) attack can be launched by sending too many legitimate messages. Further, these methods use complimentary approaches like generating probability distributions from individual STMs to detect few attack types. Taking motivation from this, we describe a new state transition model to detect cyber attacks in smart-grid networks. This new state machine can not only flag unusual sequences of events but also has different types constraints like timing, number of events of particular type, application message values, etc within one STM which improves the chances of attack detection, also help reducing the false alarms and minimizes the overhead considerably. In this direction, this paper makes the following meaningful contributions.

1. We identify three types of anomalies and show that many different cyber attacks can be detected by identifying these three types of anomalies.

Manuscript received 10 August 2023; revised 23 December 2023 and 1 March 2024; accepted 2 March 2024. Date of publication 6 March 2024; date of current version 12 July 2024. This work is financially supported by funding through a project funded by C3i-Hub IIT Kanpur via grant number IHUB-NTIHAC/2021/01/24 and also by Science and Engineering Research Board via grant CRG/2022/005198-G. The associate editor coordinating the review of this article and approving it for publication was M. J. Khabbaz. *(Corresponding author: Neminath Hubballi.)*

The authors are with the Department of Computer Science and Engineering, Indian Institute of Technology Indore, Indore 452020, India (e-mail: neminath@iiti.ac.in).

Digital Object Identifier 10.1109/TNSM.2024.3373881

1932-4537 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

uthorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 01,2024 at 17:05:14 UTC from IEEE Xplore. Restrictions apply

2. We describe a new state transition machine model DCTA which is capable of identifying the three anomaly types in SCADA communications.

3. We use extended network flow records to construct these DCTAs for detecting different anomalies.

4. We use a publicly available dataset to evaluate the performance of DCTA and compare it with prior works.

II. Related Work

We review the existing literature on detecting cyber attacks in SCADA networks under three categories as follows.

1) *Machine Learning-based Models:* Machine learning (ML) models are built to detect anomalies in smart-grids either using i) readings from Remote Terminal Units (RTUs) or ii) network data or flow/packet level details [8]. They are trained with selected features like voltage, current, angle, phase values, etc for the first type while they may be network traffic volume, source and destination IP addresses, and packet payload content, etc for the second type. Major ML techniques include:
   a) *Classification:* These methods use samples of normal and attack scenarios to train classification algorithms. Several classification algorithms like decision tree [13], random forest [13], [14], and support vector machines [13], [14], [15], one class support vector machines [16], [17] have been used for detecting anomalies in SCADA systems using network traffic. Decision tree and random forest algorithms generate a tree structure using a set of features extracted from network traffic. Support vector machines draw a decision boundary to perform the classification. Anwar et al. [18] propose to improve detection performance by extending attributes (use both static and dynamic features).
   b) *Clustering:* These methods use different algorithms to group samples that are similar and separate those which are dissimilar. K-means clustering [15], [19] and DBSCAN [20] algorithms are used to detect different types of anomalies. K-means clustering algorithm partition the dataset into several groups using phase-wise division while DBSCAN increases the size of groups/clusters by merging nearby and similar groups. Attacks are identified as outliers or smaller clusters in this case which may generate false alarms as they may pertain to legitimate communications.
   c) *Deep Learning:* Recent works [21], [22] use deep learning and convolution neural network models for detecting cyber attacks in SCADA systems. These models can automate the feature extraction to some extent. However, these models work as black boxes without providing adequate details for the feature selection and performance.

   Further, ML methods require a complete and representative dataset for training [14] and also these methods suffer from false alarms potentially reducing the overall efficiency of the security system [13]. Owing to these issues; work of Suaboot et al. [23] advocates the use of ensemble-based techniques like Voting and Boosting to enhance accuracy. The study also recommends integrating automated detection with rule-based solutions for flexibility in identifying unknown malicious incidents. However, Teryak et al. [24] show that ML based algorithms are susceptible for adversarial attacks and their accuracy drops significantly under adversarial attacks. In order to improve the detection performance under such attacks, they adopt a hierarchical multi-layer perceptron (HMLP) and also use defensive distillation to improve the model's resilience.

2) *State Transition Models:* State transition modeling is a method where the operations of smart-grid is modeled as a set of states and transitions between them. These machines can represent the normal operation or capture specific error conditions. Using these machines; anomalies can be detected in either of these ways. i. when a sequence of moves which are not seen before are identified. ii. when the sequence of moves is rare as estimated by its probability. Cheung et al. [10] were the first ones to model the Modbus operation as a State Transition Model (STM) to identify anomalies. Goldenberg and Wool [11] model the Modbus operation as a deterministic finite automata. This is based on the premise that Modbus messages are sent at periodic intervals. Hence they exhibit a high degree of periodicity. The STM is then used to identify anomalous behavior in the communication, indicating a potential intrusion. Caselli et al. [12] modeled the sequential communication of Modbus as Discrete Time Markov Chain (DTMC). However, they observed that several of the communications do not clearly exhibit cyclic behavior as assumed by Goldenberg and Wool [11], and only 35%-70% of the transitions were seen due to the multiplexing of several communications over a single TCP connection and asynchronous behavior mode of communication. This is addressed in a followup work [25] by having specific DFAs for every message/command sequence. A global DFA-Selector will select appropriate DFAs for every message by splitting the traffic of different channels and sending them to appropriate DFAs. Further the SACDA traffic is observed to be bursty in nature [26] where several packets appear together and one packet to subsequent packet is not equally spaced. Thus they presented a new burst DFA model to capture this behavior. Mustafa et al. [27] described a specification based IDS to augment the DFA to improve the detection performance.

   The closest work to ours is of Matoušek et al. [9] which proposed a method for modeling Industrial Control Systems (ICS) communications using their probability/frequency values as an automata. Deterministic Probabilistic Automata (DPA) and Prefix Tree automata are the two automata types used in this work. While DPA generates a probabilistic automata with every transition assigned a probability, Prefix Tree assigns frequency values (natural

uthorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 01,2024 at 17:05:14 UTC from IEEE Xplore. Restrictions apply

number) to every transition. Here the traffic between two devices is seen as conversation and every conversation is a string with a frequency/probability of occurrence. Frequencies of transitions in `Prefix Tree` is the number of times a particular conversation is found in the training sequences. Alergia [28] learning algorithm is used to generate `Deterministic Probabilistic Automata (DPA)` which assigns probability values to different transitions. These transition probabilities are generated by dividing the individual transition frequency values by the overall frequency of all outgoing transitions and the frequency of acceptance of a state. Thus every transition probability contributes to the probability of a string. Alergia makes a few more optimizations by merging states to reduce the size of `DPA`.

Most of the state transition models described only capture the sequence of messages exchanged and hence can not detect timing and other types of anomalies like connection loss.

3) *Other Methods:* Several other works explored detecting cyber attacks in smart-grid networks with signature based intrusion detection systems [29], behavior-based models [30], etc. Signature based detection systems identify attacks using a set of rules. On the other hand behavioral and statistical models detect deviations from known normal behavior profiles. Statistical methods such as control charts [31] are used to monitor the variability in system behavior and detect anomalies that fall outside the established control limits. Similarly regression analysis [32] can identify relationships between different variables and identify patterns that do not follow the established communication patterns. Another common method used for detecting anomalies in smart-grid networks is using time-series analysis [33], [34] which can detect anomalies by analyzing the pattern of data over time. A recent work [35] proposes to model the sensor/actuator driven communication which is not solicited by the control center to identify Advanced Persistent Threats (APTs) and malware infections. They use Multi Variate Correlation technique to detect such anomalies.

III. Proposed Anomaly Detector

In this section, we describe the proposed state transition model `Deterministic Counting Timed Automata (DCTA)` which detects cyber attacks using network flow records of IEC 60870-5-104 communication. Hence we begin by providing a brief overview of IEC 60870-5-104 communication protocol, details of different attack vectors considered and subsequently, we formally define the `DCTA` and also show the state transition models designed for detecting these attacks.

A. IEC 60870-5-104 Communication Background

IEC 60870-5-104 protocol stack is a standard developed by International Electrotechnical Commission (IEC) for telecontrol of equipment/system over TCP/IP.[1] The standard defines a message format in the form of Application Protocol Data Unit (APDU) and these messages are used for communication between the master station (SCADA master) and the remote devices (slaves). APDU is made up of a mandatory Application Protocol Control Information (APCI) or an APCI with Application Service Data Unit (ASDU) as shown in Figure 1.

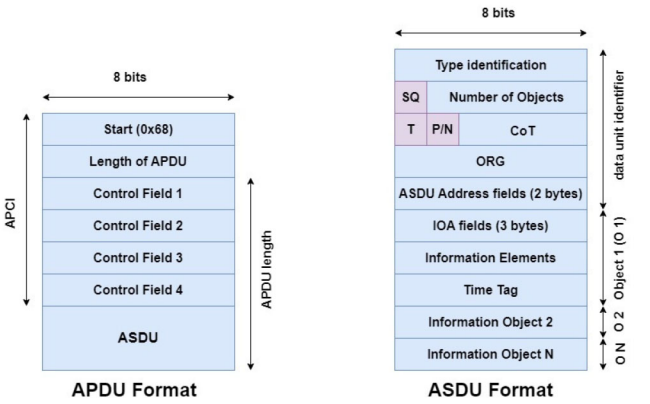


Fig. 1. APDU Message Format.

The control information that is added to the payload data is referred to as APCI. It consists of functions like start, stop, test, reset, and data transfer which enables session establishment, termination, reliable data transmission, and link integrity checks in the protocol. ASDU consists of data identifier field and data itself. Data identifier field includes ASDUTYPE, Number of Information Objects (NUMIX) within APDU, Cause of Transmission (CoT), Originator Address (ORG), i.e., sender address, and ASDU address field (ADDR). ASDUTYPE represents the type of information being transmitted within an ASDU. CoT provides information about the purpose or event that triggered the transmission, allowing the recipient to understand the context or significance of the message. Data is made up of one or more information objects. IOA represents the address of information objects present inside the ASDU. It helps to identify the particular data within a defined station. The values set in the fields of APDU, number of APDU messages, and their order can help identify anomalies. For e.g., if there is a series of messages received with ASDUTYPE 46 and CoT values 6, 7, and 10 by a device, it is possibly an indicator of a switching attack (details in the next subsection).

B. Attack Vectors

There are several cyber attacks that can be launched against the SCADA infrastructure. These attacks can manifest from different layers of SCADA systems like application interfaces, Web interfaces, implementation bugs, etc. [36]. However, these cyber attacks are applicable to conventional networks as well and are well studied. We consider a few major cyber attacks which are exclusive to smart-grid networks and study their behavioral observations. We also identify three network

[1]More details in [4].

uthorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 01,2024 at 17:05:14 UTC from IEEE Xplore. Restrictions apply

anomaly types.[2] Identifying these anomalies help in detecting the attacks.

i) Connection Loss: This is an adverse situation in which a network's connection is lost due to external interruptions which include node capture, physical issues like damaged network cables, etc. As SCADA communications are periodic in nature, such interruptions result in missing communications or in general missing flows which is a pattern of failed data transmission from the controller/master to the Remote Terminal Units (RTUs) and other way around.

ii) Denial of Service: In this attack, an attacker overwhelms the master by sending a large number of messages. This keeps the master busy and may even bring it down disturbing the legitimate communications with the master. Indications of a DoS attack in smart-grids include the unresponsiveness of controller/master or RTUs, significant degradation in system performance, and a sudden increase in network traffic or requests.

iii) Injection Attack: In this attack an adversary sends spurious messages/communication to the master/controller possibly using a new device or compromising an existing device in the network. These messages can be of two types

(a) Attributes of messages having illegitimate values: Here attributes take values out of their permitted ranges. For e.g., attribute CoT can take values 6, 7, 8, 9, 10, 44, 45, 46, and 47 when attribute ASDUTYPE has values in the range 45-51. If the CoT attribute takes the value 5 when ASDUTYPE is 46, it is illegitimate.

(b) Attributes having legitimate values; but their sequence and numbers are illegitimate: Here all the attributes take legitimate values but their sequencing in a limited span of time can create illegitimate actions.

Indications of an injection attack in smart-grids include unexpected or unauthorized commands sent to RTUs, abnormal control actions or system behavior, and anomalies in data readings or sensor outputs.

iv) Rogue Devices: Rogue devices are unauthorized devices that send illegitimate messages to the master as in the case of an injection attack. However, the only difference is; these messages may be sent by known devices in the injection attack. Indications of rogue device communications include the execution of unauthorized commands, abnormal system behavior, and anomalies in data readings.

v) Scanning Attack: In this case, an attacker scans the SCADA network for potential machines/devices to communicate. Horizontal scanning and vertical scanning are two types of scanning techniques. Indications of horizontal scanning attack include an increased volume of network traffic, attempts to identify and connect to devices, and a surge in unauthorized connection requests. Indications of vertical scanning attack include attempts to connect to different Information Object Addresses (IOA) within a device. Every such object within a device is assigned an address and by sending a probe to that address, an attacker is finding if that address is active or in use. Both horizontal and vertical scanning techniques will be probing to either identify devices and addresses valid and also possibly vulnerabilities in communication protocols or device configuration.

vi) Switching Attack: In this case, an attacker sends a series of messages with specified ASDUTYPE and CoT values to the target that causes the device to turn on and off. Unauthorized changes in the control settings, abnormal switching operations or patterns, and discrepancies between commanded and actual switching states are indications of switching attack.

It is worth noting that, the above list of cyber attacks is not exhaustive. However, these and many other potential cyber attacks fall into one of the following three types of anomalies.

(i) Single Message Anomaly: These anomalies are consequence of illegal/changed values of attributes in a single message. The APDU message can have attributes set to values that are invalid. Some of the rogue communications, and injection attacks fall under this category as they involve messages being sent from unknown/illegitimate sources.

(ii) Message Sequencing Anomaly: These anomalies are caused by message sequencing that are not seen before. For e.g., the master and RTU communications carry messages with specific values set. Every response message should be preceded by a query message. Moreover, there should be a correspondence between the query and response which are usually matched with attribute values. Switching attack falls under this category where messages are sent with toggling mode of operation with a sequence between them.

(iii) Time based Anomaly: These anomalies are caused by the repeated appearance of a large number of messages over a period of time. These messages may possibly have legitimate values set for their attributes. However, their sheer number which is otherwise not seen or absence of these messages indicates an anomaly. DoS attacks fall under this category when the messages sent have legitimate values.

C. State Transition Model Based Detection

Our detection model uses extended network flows for detecting different attacks in the smart-grid networks. Corresponding to every attack, there is a state machine as shown in Figure 2. Network flows are generated from packets (collected from switch/router through which these packets pass through) with additional details as required for detection. Such flows are given as input to all the machines and all of these machines process the flow if it is relevant and take appropriate transitions depending on their current state and transition constraints. An attack is detected if any of these machines move to a state representing an attack.

In order to detect the above cyber attacks, we propose a new state transition model. This model uses network communication flows as input and performs state changes to detect the attacks. This new state transition machine, we name as `Deterministic Counting Timed Automata (DCTA)`. The machine is formally denoted as seven tuple

M = (Q, Σ, C, $\mathcal{T}$, δ, q_0, F) where

Q A finite set of states

[2]There are other types of anomalies that can be seen in SCADA systems. For e.g., measurement anomalies [37].

uthorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 01,2024 at 17:05:14 UTC from IEEE Xplore. Restrictions apply

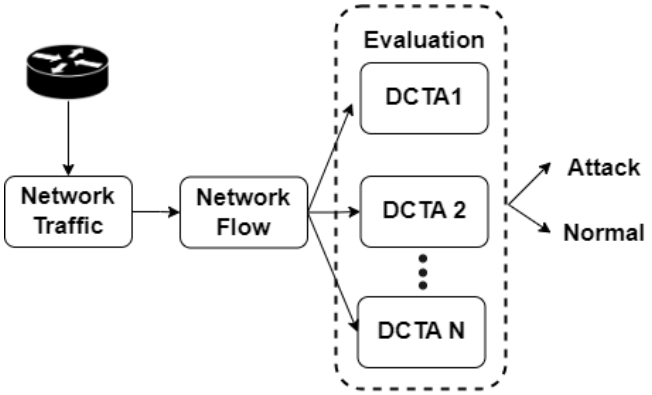


Fig. 2. Working of Proposed Detection Method.

$q_0 \in Q$ An initial state
Σ A finite set of input symbols corresponding to different messages exchanged in the SCADA system with $m_i \in \Sigma = \{a_{i1}, a_{i2}, \ldots, a_{ik}\}$ are the attributes of m_i
C A finite set of counter variables
$\mathcal{T}$ A finite set of clock variables
δ Is a transition function
F A subset of states $(F \subseteq Q)$ which are final states

Each transition $\delta_i \in \delta$.[3] is a nine tuple $\langle q_i, q_j, m_i, \phi(m_i), \phi(t_i), \phi(c_i), Reset(t_j), Inc(c_j), Log(m_i)\rangle$ with the elements representing

$q_i \in Q$ Current State
$q_j \in Q$ Next State
m_i Input Symbol
$c_i \in C$ Counter value at the state q_i
$\phi(m_i)$ Boolean conjunction of constraints on the attributes of message m_i
$\phi(t_i)$ Timing constraint on the timer at q_i
$\phi(c_i)$ Counter constraint, where c_i is the counter at q_i
$Reset(t_j)$ A function which resets the time variable $t_j \in \mathcal{T}$ at state q_j
$Inc(c_j)$ Is a function that maps the current state of the counter at q_j to a new value
$Log(m_i)$ Is a function that logs the necessary attributes from message m_i.

A value of '$-$' in a transition signifies that the corresponding value or constraint is not applicable to this transition. For example, a transition

$(q_1, q_2, m_1, LEN \in [1, 10], t_1 < 3, c_1 > 3, t_2 \leftarrow 0, c_2 \leftarrow 1, -)$

indicates a transition from state q_1 to q_2 upon arrival of a message m_1 and following constraints are met

1) one of the attributes namely length is having a value between 1 and 10,

2) message is received in less than 3 units since the last transition to q_1 and clock reset timing,

3) counter value at q_1 is greater than 3 when the transition takes place,

4) counter at state q_2, i.e., c_2 is initialized to 1 and a '$-$' at the end indicates nothing from this message m_1 is logged for subsequent usage/reference.

DCTA has few states marked as anomaly indicators or final states. Progression to these states through a sequence of moves indicate identified anomalies in the system. These sequence of moves are of the form $(q_o, m_1, q_i), (q_i, m_2, q_j), \ldots, (q_{n-1}, m_n, q_n)$ with $q_n \in F$ (to the anomaly state) also define the language accepted by the respective DCTA.

D. State Transition Models for Different Attacks

Using the state transition model presented previously, we design specific machines for detecting different attacks.

Connection-loss Attack: As mentioned earlier, communication in SCADA systems is usually periodic in nature with master querying the RTU and other measurement units for readings at regular intervals. Thus a series of such missing flows indicate connection loss and this needs to be traced through the state transition machine for detecting the attack. In order to detect this attack, we design a STM as a time based anomaly detector. The STM is initialized afresh for every *W* units of time and it keeps track of number of missing communication flows with the help of a counter within that *W* time period. Suppose, on an average after every *x* seconds one flow is received, state transition machine keeps checking after every *x* seconds (taking into account random network delays) if there is a flow or not. The STM for detecting this attack is a three state machine as shown in Figure 3. State q_1 is the initial state and q_2 is the state where counting is done. The machine stays in state q_1 if there is a message m_i (through a network flow) observed as expected; otherwise it takes a transition to q_2, increments missing flow/communication counter at q_2 and returns back to q_1.[4] The transitions from q_1 to q_1 and q_1 to q_2 are having timing constraints to indicate respective time delays. It moves to state q_3 if the counter value at q_2 exceeds a threshold which is indicated as a constraint on the transition from q_2 to q_3.

DoS Attack: DoS attack results in surge of messages/flows from an adversary. We design a time based anomaly detector with two states as shown in Figure 4 to detect this attack.

The STM is initialized afresh for every *W* units of time and it keeps track of number of incoming flows within that *W* time period. If the number of such messages increase beyond a threshold value, the STM declares attack. The state q_1 keeps track of number of flows within *W* time period by incrementing a counter if there is a message m_i observed as expected. It moves to state q_2 if the counter value at q_1 exceeds a threshold which is indicated as constraint $c_1 \geq \lambda_2$ on the transition from q_1 to q_2.

Injection Attack and Rogue Devices We observe that the actions in injection and rogue device communications are similar where some unauthorized communication or unexpected messages are transferred to the master. The only difference is that these messages may originate from a legitimate device in the injection attack and from an illegitimate device in the rouge

[3]Not a total function. However the transitions are deterministic.

[4]This is an instantaneous transition without any delay.

uthorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 01,2024 at 17:05:14 UTC from IEEE Xplore. Restrictions apply

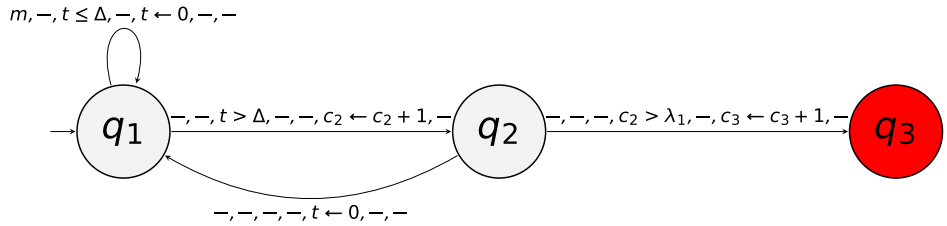


Fig. 3. DCTA for Detecting Connection Loss Attack.

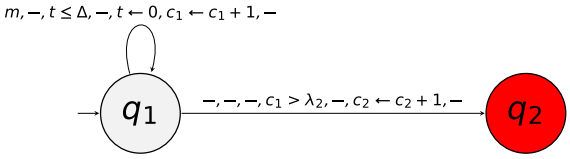


Fig. 4. DCTA for Detecting DoS Attack.

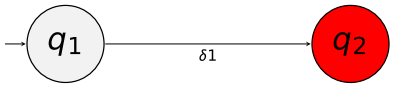


Fig. 5. DCTA for Detecting Injection/Rogue Device Attack (Single Message Anomaly).

communication. Hence, we have common STMs for detecting these two attacks. As there are two types of anomalies these attacks can generate, we have designed two types of STMs for detecting these attacks as detailed below.

(a) APDU messages having illegal values can be detected with appropriate constraints placed on the message attributes. The STM for detecting these illegal value combinations is shown in Figure 5 which is a two state STM with a single transition $\delta 1$ having constraints on message attributes as shown in Table I. For e.g., DCTA has constraint that the length of APDU (*LEN*) should be within the range 0-255 on this transition from q_1 to q_2. It is easy to see that this STM can detect single message anomaly type.[5]

(b) Second category of anomalies can be detected by looking at the sequence of messages over a very short period of time. Hence, we design a sequence based anomaly detector (STM) with appropriate constraints on transitions as shown in Figure 6. The transitions in this STM have constraints on the message type, their values, timing, etc as shown in Table I. Here, STM keeps track of legitimate devices as well as values of different attributes of messages by logging these using function *Log*(*m*). Message constraint imposed on transitions $\delta 1$ and $\delta 2$ is a collection of constraints on various attributes of a message. A control command ASDUTYPE takes values 45, 47, 48, 49, 50, and 51 except for the double point control command (ASDUTYPE $=$ 46) which is used to switch on/off a device. Moreover, these messages should come from a legitimate device (known IP address). Thus, the message constraint imposed on transition $\delta 1$ denotes that only legitimate device IP addresses should be used. In addition to this, no such message should appear in which the control command activation and confirmation are happening sequentially within a limited amount of time. Similarly, the constraint on transition $\delta 2$ denotes that if control command activation and confirmation are done sequentially within a limited amount of time then it represents an anomaly. Whenever devices apart from the legitimate ones try to communicate; then it gets classified as a rogue device.

Scanning Attack: In the case of a scanning attack, the attacker sends specific messages to scan the network to find active devices. If any device is active, then the scan yields a response in the form of test frame confirmation. In order to detect this attack, we designed a STM as a time based anomaly detector with two states as shown in Figure 7. The STM is initialized afresh for every *W* units of time and it keeps track of the number of messages sent to different devices or different objects within a device. If the number of such messages increases beyond a threshold value, the STM detects an attack.[6] The state q_1 keeps track of the number of such messages sent for scanning within *W* time period by incrementing a counter if there is a message m_i observed such that it is addressed to a different device (different IP address in case of a horizontal scanning) or different objects (different Information Object Addresses) within a device (vertical scanning). IP addresses of devices that received messages and also the Information Object Addresses of objects within a device are tracked by logging the details using function *Log*(*m*). It moves to state q_2 if the counter value at q_1 exceeds a threshold which is indicated as constraint $c_1 \geq \lambda_3$ on the transition from q_1 to q_2. The transitions corresponding to this STM are shown in Table I.

Switching Attack: In switching attack, attacker sends multiple messages with ASDUTYPE value 46 and changing the CoT values to 6, 7 and 10. Thus, detecting this attack requires analyzing a sequence of messages for these changes. In order to capture these changes, we designed a STM as a time based anomaly detector with two states as shown in Figure 8. The transition from the state q_1 to q_1 keeps track of different attribute values (CoT and ASDUTYPE extracted from messages) by logging them using function *Log*(*m*). It moves to state q_2 if the earlier logged values (within a short window period) contain CoT value 6, 7, 10 and ASDUTYPE value 46, as it indicates acceptance of switching command immediately which can bypass normal control system logic and can override established protection schemes. The transitions corresponding to this STM are shown in Table I.

[5]As there are many combinations of values, we show a few representative constraints here and omit the rest of the details for the sake of brevity.

[6]It will keep track of only either of these two.

uthorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 01,2024 at 17:05:14 UTC from IEEE Xplore. Restrictions apply

TABLE I
TRANSITIONS OF DIFFERENT DCTAS

Transition	Current State	Next State	Input Symbol	$\phi(m)$	$\phi(t)$	$\phi(c)$	Reset(t)	Inc(c)	Log(m)
DCTA Transitions of Figure 5 for Injection/Rogue Device Attack (Single Message Anomaly) Detection									
$\delta 1$	q_1	q_2	$m\,(APDU)$	$srcPort, dstPort \notin [0, 65535]$ || $LEN \notin [0, 255]$ || $FMT \notin 0x0, 0x1, 0x3$ || $ASDUTYPE \notin [1, 135]$ || $NUMIX \notin [0, 127]$ || $CoT \notin [1, 47]$ || $ADDR \notin [1, 65535]$ || $IOA \notin [0, 65535]$ || $(ASDUTYPE = 35$ && $\sim (CoT = 3$ || $CoT = 5))$	–	–	–	–	–
DCTA Transitions of Figure 6 for Sequence based Injection Attack and Rogue Devices Detection									
$\delta 1$	q_1	q_1	$m\,(APDU)$	$srcIP, dstIP \in Log(m)$ && $\sim (\{ASDUTYPE = 45$ && $CoT = \{6, 7\}\} \in Log(m)$ || $\{ASDUTYPE = 47$ && $CoT = \{6, 7\}\} \in Log(m)$ || $\{ASDUTYPE = 48$ && $CoT = \{6, 7\}\} \in Log(m)$ || $\{ASDUTYPE = 49$ && $CoT = \{6, 7\}\} \in Log(m)$ || $\{ASDUTYPE = 50$ && $CoT = \{6, 7\}\} \in Log(m)$ || $\{ASDUTYPE = 51$ && $CoT = \{6, 7\}\} \in Log(m)$ ||)	–	–	–	–	Log(m)
$\delta 2$	q_1	q_2	$m\,(APDU)$	$srcIP, dstIP \notin Log(m)$ || $\{ASDUTYPE = 45$ && $CoT = \{6, 7\}\} \in Log(m)$ || $\{ASDUTYPE = 47$ && $CoT = \{6, 7\}\} \in Log(m)$ || $\{ASDUTYPE = 48$ && $CoT = \{6, 7\}\} \in Log(m)$ || $\{ASDUTYPE = 49$ && $CoT = \{6, 7\}\} \in Log(m)$ || $\{ASDUTYPE = 50$ && $CoT = \{6, 7\}\} \in Log(m)$ || $\{ASDUTYPE = 51$ && $CoT = \{6, 7\}\} \in Log(m)$ ||	–	–	–	–	–
DCTA Transitions of Figure 7 for Horizontal Scanning Detection									
$\delta 1$	q_1	q_1	$m\,(APDU)$	$dstIP \notin Log(m)$	$t \leq \Delta$	–	$t \leftarrow 0$	$c_1 \leftarrow c_1 + 1$	$Log(m)$
$\delta 2$	q_1	q_2	$m\,(APDU)$	–	–	$c_1 > \lambda_3$	–	$c_2 \leftarrow c_2 + 1$	–
DCTA Transitions of Figure 7 for Vertical Scanning Detection									
$\delta 1$	q_1	q_1	$m\,(APDU)$	$IOA \notin Log(m)$	$t \leq \Delta$	–	$t \leftarrow 0$	$c_1 \leftarrow c_1 + 1$	$Log(m)$
$\delta 2$	q_1	q_2	$m\,(APDU)$	–	–	$c_1 > \lambda_3$	–	$c_2 \leftarrow c_2 + 1$	–
DCTA Transitions of Figure 8 for Switching Attack Detection									
δ_1	q_1	q_1	$m\,(APDU)$	–	$t \leq \Delta$	–	$t \leftarrow 0$	–	$Log(m)$
δ_2	q_1	q_2	$m\,(APDU)$	$\{ASDUTYPE = 46$ && $CoT = \{6, 7, 10\}\} \in Log(m)$	–	–	–	–	–

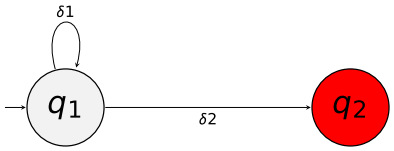


Fig. 6. DCTA for Detecting Sequence Based Injection Attack and Rogue Devices.

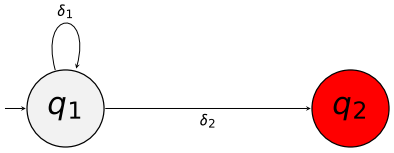


Fig. 7. DCTA for Detecting for Scanning Attack.

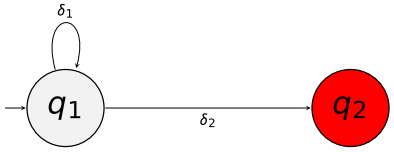


Fig. 8. DCTA for Detecting Switching Attack.

Attack Detection with DCTAs an Example: Consider that a fresh window starts at w_1 and lasts upto w_2. All the packets collected from a router/switch in this period are used to construct flows with necessary additions. Lets assume an attacker with a valid IP address is trying to perform scanning attack by sending legitimate packets but with different *IOA* values to check for an active victim. It sends IEC 104 packets featuring an Information Object Address (*IOA*) denoted as x_1. Flow generation will generate a flow corresponding to this which will eventually fed to all DCTAs. In response, all DCTAs governing distinct security aspects react concurrently. The DCTA of connection loss attack reverts to its initial state (q_1 to q_1), recognizing the presence of the incoming packet. Similarly, the DCTA monitoring rogue and injection activities maintains its initial state, as the packet is validated as legitimate (IP address is valid). The Denial of Service (DoS) DCTA increments its counter, acknowledging the presence of a fresh packet. The DCTA, designed for detecting scanning activities, logs the *IOA* value if the value is absent in the log file otherwise it increments its counter only. The Switching DCTA, checking for specific *CoT* and *ASDUTYPE* values remains in the initial state as there are no values corresponding to this in the flow/packet. In the event of receiving a sequence of packets with diverse *IOA* values (prior to this packet/flow which are logged previously), the scanning DCTA may transition to a state indicating an attack if the counter surpasses a predefined threshold. Subsequently, at the end of each window period, a reset occurs across all DCTAs, ensuring a clean slate (including clearing logged values) for subsequent monitoring and analysis.

IV. EXPERIMENTS AND EVALUATION

In this section, we provide the details of experiments done to evaluate the detection performance of DCTA based anomaly detectors. We provide the details of evaluation results and sensitivity analysis in the following two subsections.

A. Evaluation

Here we describe the dataset used, implementation details, comparison with prior works, evaluation metrics and evaluation results respectively.

uthorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 01,2024 at 17:05:14 UTC from IEEE Xplore. Restrictions apply

TABLE II
OVERVIEW OF DATASET

Type	Duration	Flow Count
Normal-traffic	67 hours 55 minutes	58,930
Connection-loss	67 hours 55 minutes	57,863
DoS-attack	67 hours 55 minutes	58,932
Injection-attack	67 hours 55 minutes	58,930
Rogue-device	67 hours 55 minutes	58,893
Scanning-attack	67 hours 55 minutes	58,927
Switching-attack	67 hours 55 minutes	59,002

Dataset: For our experiments and evaluation, we used a publicly available dataset [38], namely but-iec104-i collected from a smart-grid testbed at Brno University of Technology. This dataset is a processed collection of network flow records (in csv format) of IEC 104 communications. These flow records are extended IPFIX flows with a few custom fields added which indicate the type of IEC messages appeared in the corresponding flow. These flows are collected from a network device that observes the network traffic and generates IPFIX records and exported to a flow collector. Subsequently, these flows are processed and annotated after custom field additions. The records include fields like timestamps, IP addresses, port numbers of communicating devices, and also a few header details taken from IEC 104 messages, etc. The dataset consists of normal communication and also six types of cyber attacks generated at different time intervals. Table II shows the details of this dataset.

The dataset consists of seven files each of a duration of 67 hours and 55 minutes. One of them is of normal communication and the other six correspond to an attack type. The table also shows the number of flow records in each of these files.

Implementation: We implemented DCTAs for detecting different anomalies/attacks in Python programming language. STMs have two main components namely states and transitions. In our implementation, we represented all the states with variables, and the current state is remembered by storing the state name in a separate variable. Transitions of the DCTA are implemented through a function named `transition()` which takes the current state and different constraints as input. We enlisted the different constraints (message attribute, timing, and counter values) that cause the transition from one state to another. For every state, we defined all of the possible transitions and the corresponding next state of the machine. In the function `transition()`, we implemented logic to evaluate the different constraints with *if-then-else* ladder to make an appropriate state change in STM. For every state, all the transitions from that state are attempted to check if a move is possible. At the end of every transition, the current state value is updated. It is also possible that the current state is same as the previous state as some of the STMs have self loops.

Comparison with Prior Work: We compare the performance of our proposed DCTA based anomaly detector with two recent works namely of Matoušek et al. [9] which described two different types of STMs to detect anomalies and also with Teryak et al. [24].

First one identify two types of anomalies.

TABLE III
GOLDEN THRESHOLD VALUES

Parameter	Threshold
λ_1	30
λ_2	45
λ_3	27
Δ	8

1) Single Conversation Reasoning- Using an STM constructed from the sequence of messages, it identifies transitions which were never seen before (i.e., the traces from which STM is generated). Such transitions are detected as anomalies.

2) Comparing the Probability Distributions- All conversation sequences (message sequences in the trace) are used to generate `DPA/Prefix Tree` type STMs and subsequently these STMs are used to derive frequency values of individual transitions. These transition frequency values are represented as probability distributions indicating the likelihood of a particular transition. Subsequently, the probability distributions of five minute window periods from the training and testing intervals are compared with Euclidean distance. If the distance is high, then an attack is detected.

On the other hand, Teryak et al. [24] use hierarchical multilayered perceptron based ML algorithm to improve detection performance. Implementation for both the methods are available and results reported here are based on their code.

Evaluation Metrics: We evaluate the performance of the proposed DCTA based anomaly detector with five parameters namely `Detection Rate (DR)`, `False Positive Rate (FP)`, `Accuracy (Acc)`, `F1 score (F1)` and `Matthews Correlation Coefficient` (MCC).

Further, it is worth noting that we use few thresholds as constraints for transitions of DCTA. Few of these transitions lead to states representing attacks/anomalies. The parameters and corresponding threshold values are shown in Table III which we denote as golden thresholds.

Results: We present the results of the evaluation in two parts.

`(i) Grand Experiment:` Here we assess the performance on the combined dataset where all seven files (one normal and six attack types) are merged. For those attack types which are of time based anomalies, we used a window time of five minutes. Entire dataset was given as input to both DCTA based STMs, also the two STMs of [9] and to the HMLP method to evaluate the performance. Table IV. shows the results of this experiment with different metrics. We can notice that DCTA exhibits far better performance with 100% detection rate, nearly 100% accuracy, and F1 scores while both the methods of prior work have paired comparatively poorly. The low detection rate of `Prefix Tree` and DPA based learning methods is due to their inability in detecting certain attack types (details in the next part).

`(ii) Individual Experiment:` In the second part, we performed an evaluation with individual attack types combined with normal communication flows and compare it with only [9] as this is STM based model. Thus there are six files each having 1630 intervals of five minutes duration. The objective of this evaluation is to assess how good the detection is for different attacks. For this experiment also we used the

uthorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 01,2024 at 17:05:14 UTC from IEEE Xplore. Restrictions apply

TABLE IV
DETECTION PERFORMANCE COMPARISON OF DCTA WITH PRIOR WORKS ON GRAND DATASET

Testing Intervals	DCTA				Reference [9]								HMLP [24]			
					Prefix Tree				DPA							
	DR	FP	Acc	F1	DR	FP	Acc	F1	DR	FP	Acc	F1	DR	FP	Acc	F1
5705	100%	0.017%	99.98%	99.29%	47.14%	0%	99.35%	64.08%	47.14%	0%	99.35%	64.08%	100%	0.6%	99.4%	94.97%

TABLE V
DETECTION PERFORMANCE COMPARISON OF DCTA WITH PRIOR WORKS ON INDIVIDUAL ATTACK TYPES

DCTA								DPA								Prefix Tree							
Confusion Matrix			DR	FP	Acc	F1	MCC	Confusion Matrix			DR	FP	Acc	F1	MCC	Confusion Matrix			DR	FP	Acc	F1	MCC
	Normal	Connection-loss							Normal	Connection-loss							Normal	Connection-loss					
Normal	1616	0	100%	0%	100%	100%	1	Normal	1616	0	85.71%	0%	99.87%	92.30%	0.92	Normal	1616	0	85.71%	0%	99.87%	92.30%	0.92
Connection-loss	0	14						Connection-loss	2	12						Connection-loss	2	12					
	Normal	DoS							Normal	DoS							Normal	DoS					
Normal	1595	0	100%	0%	100%	100%	1	Normal	1595	0	0%	0%	97.85%	0%	0	Normal	1595	0	0%	0%	97.85%	0%	0
DoS	0	35						DoS	35	0						DoS	35	0					
	Normal	Scanning							Normal	Scanning							Normal	Scanning					
Normal	1623	0	100%	0%	100%	100%	1	Normal	1623	0	100%	0%	100%	100%	1	Normal	1623	0	100%	0%	100%	100%	1
Scanning	0	7						Scanning	0	7						Scanning	0	7					
	Normal	Switching							Normal	Switching							Normal	Switching					
Normal	1627	1	100%	0.061%	99.94%	80%	0.82	Normal	1628	0	100%	0%	100%	100%	1	Normal	1628	0	100%	0%	100%	100%	1
Switching	0	2						Switching	0	2						Switching	0	2					
	Normal	Injection							Normal	Injection							Normal	Injection					
Normal	1624	0	100%	0%	100%	100%	1	Normal	1624	0	100%	0%	100%	100%	1	Normal	1624	0	100%	0%	100%	100%	1
Injection	0	6						Injection	0	6						Injection	0	6					
	Normal	Rogue							Normal	Rogue							Normal	Rogue					
Normal	1624	0	100%	0%	100%	100%	1	Normal	1624	0	100%	0%	100%	100%	1	Normal	1624	0	100%	0%	100%	100%	1
Rogue	0	6						Rogue	0	6						Rogue	0	6					

TABLE VI
DETECTION PERFORMANCE OF HMLP [24]

	Benign	Connection-loss	DoS	Switching	Scanning	Rogue	Injection
Benign	199814	0	278	650		0	350
Connection-loss	0	3033	0	0	0	0	0
DoS	0	0	1243	0	118	0	79
Switching	0	0	0	1511	0	0	0
Scanning	0	0	7	0	3006	0	0
Rogue	0	2	0	0	0	1562	0
Injection	0	0	0	0	0	0	1517

TABLE VII
DETECTION ABILITY COMPARISON OF DCTA

Anomaly	DCTA	Single	$Distr_{Prefix\ Tree}$	$Distr_{DPA}$
Connection-loss Attack	✓	✗	✓	✓
DoS Attack	✓	✗	✗	✗
Scanning Attack	✓	✓	✓	✓
Switching Attack	✓	✓	✓	✓
Rogue Device Attack	✓	✓	✓	✓
Injection Attack	✓	✓	✓	✓

same golden threshold values shown in Table III. Table V shows the results obtained with DCTA, DPA, and Prefix Tree models. The table shows the confusion matrices for each experiment along with the detection rate, false positive rate, accuracy, and F1 score and MCC values.

It is worth noting from Table V that the DPA and Prefix Tree based models were unable to detect DoS attack as the messages sent during DoS attack are also legitimate ones. The probability/frequency of these transitions are well within the limit and hence are not detectable. However, DCTA is able to detect all those attacks by keeping track of number of continuous messages sent, their order, timing, and comparison with previous message attribute values as defined in their respective transitions. Further, DCTA based detection does not classify one attack type as other one. However, HMLP [24] based detection has this issue as shown in Table VI. We can notice from the table that several attacks of one type are being detected as other type. However, this does not alter the overall detection rate as long as it is detected as attack.

Based on the considered cyber attacks, we also tabulate the detection ability of different STM based detection methods. Table VII shows this comparison. In the table, Single refers to single conversation reasoning while $Distr_{Prefix\ Tree}$ and $Distr_{DPA}$ represent the probability distribution based anomaly detectors as defined in [9]. We can notice that DCTA is able to detect all the attack types while the STMs of [9] fail to detect a few attacks. In contrast, our proposed anomaly categorization aims to be comprehensive and can potentially detect different types of attacks.

B. Sensitivity Analysis

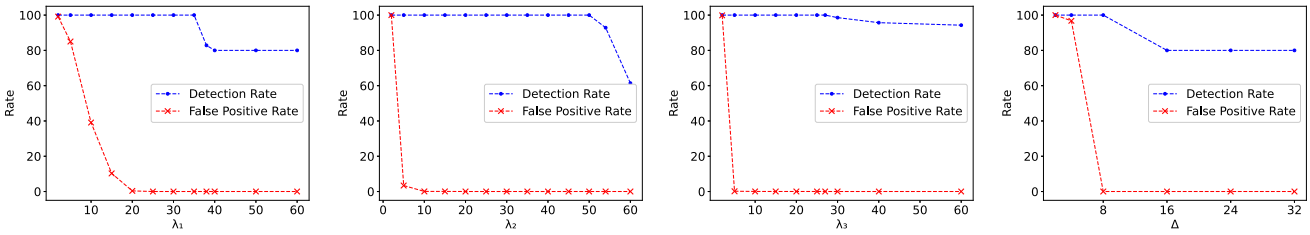
It is worth noting that, several transitions of proposed DCTA models use thresholds for counter values at different states and also timing constraints. Thus the detection performance will depend on these threshold values. In order to assess the performance variance with these threshold values, we conduct the sensitivity analysis of two parameters namely counter values and timing.

(i) Counter Values: There are three attack types namely connection loss, DoS, and scanning whose DCTAs have constraints on counter values namely $\lambda_1, \lambda_2, \lambda_3$ as shown earlier. Thus we assess the performance variation of respective DCTAs to variations in these thresholds using grand dataset. For this evaluation, we only varied one parameter value (λ_i) and kept all other parameters as in the grand experiment evaluation discussed above. The details of performance variation for different attacks are as below.

Connection Loss: The DCTA of connection-loss detection has λ_1 as a constraint on the transition from q_2 to q_3. The golden value of λ_1 is 30 which yielded 100% detection and 0% false positives. We varied the value of λ_1 in step size of 10. Figure 9(a) shows the performance variation in detection rate and false positive rates of this experiment. We can observe from the figure that for a threshold value around 20, false positives are completely reduced to 0% and the detection rate is maintained at 100% till the threshold value crosses 30. Hence a value in this range for λ_1 will yield better performance.

Denial of Service: The DCTA designed for detecting DoS attack has a counter threshold λ_2 on the transition from state q_1 to q_2 whose golden value is 45. Similar to the previous case, here also we varied the value of λ_2 in step size of 10. Figure 9(b) displays the change in detection rate and false positive rate with respect to change in the value of λ_2. We

uthorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 01,2024 at 17:05:14 UTC from IEEE Xplore. Restrictions apply



(a) Detection Performance for Different Values of λ_1 (b) Detection Performance for Different Values of λ_2 (c) Detection Performance for Different Values of λ_3 (d) Detection Performance for Different Values of Δ

Fig. 9. Detection performance variation with different thresholds.

can notice from this figure that the detection rate is 100% until the threshold crosses 45 and beyond which it falls. Further, the false positive rates fall to 0% for a value beyond 10.

Scanning: The DCTA of scanning attack has a threshold λ_3 on the transition between q_1 and q_2 whose golden threshold is 27. Similar to the above two cases, we varied this value in step size of 10 and calculated the detection rate and false positive rates. Figure 9(c) shows the variation in these values. We can notice that the detection rate is 100% till this threshold crosses 27 and the false positive rate falls to 0% to a threshold value beyond 5.

(ii) Timing Values: Several transitions of different DCTAs have timing constraints Δ. Timing constraint control when a transition is enabled. Smaller values may trigger transitions prematurely; while too large values can delay both transition and anomaly detection. In order to assess the performance variation of detection, we performed an evaluation by varying the Δ values in step size of 8. Figure 9(d) shows the variation in detection rate and false positive rates for the connection-loss attack.[7] on the grand dataset. It can be observed from the figure that too small values of Δ generate large number of false positives (due to premature transitions), while large values reduce false positives but may decrease detection performance.

V. Conclusion

Smart-grid is a critical infrastructure and it uses ICT for exchanging safety critical information and hence is vulnerable to different cyber attacks. In this paper, we presented three types of generic anomaly types namely single message based, message sequence based, and time based anomalies which help in detecting these attacks. In order to detect these anomalies (attacks), we described a new state transition based model DCTA which takes extended network flow records as input and makes permitted transitions. The DCTA transitions have different types of constraints which help verify several attribute values, sequence, timing, number of messages, etc to detect attacks. We evaluated the performance of our proposed approach on a publicly available dataset and benchmarked its performance against the prior works to show that it can detect different types of attacks with very good detection rate of 100% in the best case. It can also detect attack types which are not detected by other STM based detectors. Further, the anomaly types presented provide a guideline or template for creating DCTAs for new attack types.

DCTA based detection shares a common limitation with other STM based methods which is the requirement of a specific STM for every attack type. In addition, our work and evaluation at present is limited to only application layer attacks and as such deals with only IEC 104 protocol. However, it will be useful to consider other protocols like Modbus, DNP, etc and explore if similar models can be adopted. Further, other attack types like layer4, layer3, and layer2 of TCP/IP stack can also be combined for evaluation as they will also have an impact on the smart-grid operation.

[7]Similar results were obtained for other DCTA's and we do not show them here for space constraints.

References

[1] A. Sidhik. "Different types of SCADA protocols." forumautomation. 2023. [Online]. Available: https://forumautomation.com/t/different-types-of-scada-protocols/4194

[2] R. Mattioli and K. Moulinos (Eur. Union Agency Cybersecur., Athens, Greece). *Communication Network Interdependencies in Smart Grids*. (2016). [Online]. Available: https://www.enisa.europa.eu/publications/communication-network-interdependencies-in-smart-grids

[3] ModbusProtocol. "Modbus tools." Accessed: Jul. 27, 2023. [Online]. Available: https://www.modbustools.com/modbus.html

[4] P. Matoušek, "Description and analysis of IEC 104 protocol," Faculty of Inf. Technol., Brno Univ. Technol., Brno, Czech, Rep. TR-2017-12, 2017.

[5] J. Klick, S. Lau, D. Marzin, J.-O. Malchow, and V. Roth, "Modeling modbus TCP for intrusion detection," in *Proc. Blackhat Conf.*, 2015, pp. 1–9.

[6] R. Spenneberg, M. Brüggemann, and H. Schwartke, "PLC-blaster: A worm living solely in the PLC," in *Proc. Blackhat Aisa Conf.*, 2016, pp. 1–16.

[7] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the cyber attack on the ukrainian power grid," E-ISAC, Washington, DC, USA, White Paper, 2016.

[8] S. V. B. Rakas, M. D. Stojanović, and J. D. Marković-Petrović, "A review of research work on network-based SCADA intrusion detection systems," *IEEE Access*, vol. 8, pp. 93083–93108, 2020.

[9] P. Matoušek, V. Havlena, and L. Holík, "Efficient modelling of ICS communication for anomaly detection using probabilistic automata," in *Proc. IFIP/IEEE Int. Symp. Integr. Netw. Manage. (IM)*, 2021, pp. 81–89.

[10] S. Cheung, B. Dutertre, M. Fong, U. Lindqvist, K. Skinner, and A. Valdes, "Using model-based intrusion detection for SCADA networks," in *Proc. SCADA Secur. Sci. Symp.*, vol. 46, 2007, pp. 1–12.

[11] N. Goldenberg and A. Wool, "Accurate modeling of Modbus/TCP for intrusion detection in SCADA systems," *Int. J. Crit. Infrastruct. Protect.*, vol. 6, no. 2, pp. 63–75, 2013.

[12] M. Caselli, E. Zambon, and F. Kargl, "Sequence-aware intrusion detection in industrial control systems," in *Proc. 1st ACM Workshop Cyber-Phys. Syst. Secur. (CPSS)*, 2015, pp. 13–24.

[13] J. M. Beaver, R. C. Borges-Hink, and M. A. Buckner, "An evaluation of machine learning methods to detect malicious SCADA communications," in *Proc. 12th Int. Conf. Mach. Learn. Appl. (ICMLA)*, 2013, pp. 54–59.

uthorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 01,2024 at 17:05:14 UTC from IEEE Xplore. Restrictions apply

[14] B. Phillips, E. Gamess, and S. Krishnaprasad, "An evaluation of machine learning-based anomaly detection in a SCADA system using the modbus protocol," in *Proc. ACM Southeast Conf. (ASM)*, 2020, pp. 188–196.
[15] S. D. Anton, S. Kanoor, D. Fraunholz, and H. D. Schotten, "Evaluation of machine learning-based anomaly detection algorithms on an industrial Modbus/TCP data set," in *Proc. 13th Int. Conf. Avail., Rel. Secur. (ARES)*, 2018, pp. 1–10.
[16] J. Jiang and L. Yasakethu, "Anomaly detection via one class SVM for protection of SCADA systems," in *Proc. Int. Conf. Cyber-Enabled Distrib. Comput. Knowl. Discov.*, 2013, pp. 82–88.
[17] E. G. da Silva, A. S. D. Silva, J. A. Wickboldt, P. Smith, L. Z. Granville, and A. Schaeffer-Filho, "A one-class NIDS for SDN-based SCADA systems," in *Proc. IEEE 40th Annu. Comput. Softw. Appl. Conf. (COMPSAC)*, 2016, pp. 303–312.
[18] M. Anwar, L. Lundberg, and A. Borg, "Improving anomaly detection in SCADA network communication with attribute extension," *Energy Inform.*, vol. 5, no. 1, pp. 1–22, 2022.
[19] I. Kiss, B. Genge, P. Haller, and G. Sebestyén, "Data clustering-based anomaly detection in industrial control systems," in *Proc. IEEE 10th Int. Conf. Intell. Comput. Commun. Process. (ICCP)*, 2014, pp. 275–281.
[20] A. Almalawi, A. Fahad, Z. Tari, A. Alamri, R. AlGhamdi, and A. Y. Zomaya, "An efficient data-driven clustering technique to detect attacks in SCADA systems," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 5, pp. 893–906, 2016.
[21] H. Yang, L. Cheng, and M. C. Chuah, "Deep-learning-based network intrusion detection for SCADA systems," in *Proc. IEEE Conf. Commun. Netw. Security (CNS)*, 2019, pp. 1–7.
[22] G. Fenza, M. Gallo, and V. Loia, "Drift-aware methodology for anomaly detection in smart grid," *IEEE Access*, vol. 7, pp. 9645–9657, 2019.
[23] J. Suaboot et al., "A taxonomy of supervised learning for IDSs in SCADA environments," *ACM Comput. Surveys*, vol. 53, no. 2, pp. 1–37, 2020.
[24] H. Teryak, A. Albaseer, M. Abdallah, S. Al-Kuwari, and M. Qaraqe, "Double-edged defense: Thwarting cyber attacks and adversarial machine learning in IEC 60870-5-104 smart grids," *IEEE Open J. Ind. Electron. Soc.*, vol. 4, no. 3, pp. 629–642, Nov. 2023.
[25] A. Kleinmann and A. Wool, "Automatic construction of Statechart-based anomaly detection models for multi-treaded industrial control systems," *ACM Trans. Intell. Syst. Technol.*, vol. 8, no. 4, pp. 1–21, 2017.
[26] C. Markman, A. Wool, and A. A. Cardenas, "A new burst-DFA model for SCADA anomaly detection," in *Proc. Workshop Cyber-Phys. Syst. Secur. Privacy (CPS)*, 2017, pp. 1–12.
[27] M. Faisal, A. A. Cardenas, and A. Wool, "Modeling modbus TCP for intrusion detection," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, 2016, pp. 386–390.
[28] R. C. Carrasco and J. Oncina, "Learning stochastic regular grammars by means of a state merging method," in *Proc. Int. Colloq. Gramm. Inference (ICGI)*, 1994, pp. 139–152.
[29] R. Udd, M. Asplund, S. Nadjm-Tehrani, M. Kazemtabrizi, and M. Ekstedt, "Exploiting bro for intrusion detection in a SCADA system," in *Proc. 2nd ACM Int. Workshop Cyber-Phys. Syst. Secur.*, 2016, pp. 44–51.
[30] S. Zhanwei and L. Zenghui, "Abnormal detection method of industrial control system based on behavior model," *Comput. Secur.*, vol. 84, pp. 166–178, Jul. 2019.
[31] L. Chen and X. Wang, "Quickest attack detection in smart grid based on sequential monte carlo filtering," *IET Smart Grid*, vol. 3, no. 5, pp. 686–696, 2020.
[32] S. Bhattacharjee, M. J. Islam, and S. Abedzadeh, "Robust anomaly based attack detection in smart grids under data poisoning attacks," in *Proc. 8th ACM Cyber-Phys. Syst. Secur. Workshop*, 2022, pp. 3–14.
[33] C.-Y. Lin, S. Nadjm-Tehrani, and M. Asplund, "Timing-based anomaly detection in SCADA networks," in *Proc. 12th Int. Conf. Crit. Inf. Infrastruct. Secur. (CRITIS)*, 2018, pp. 48–59.
[34] C.-Y. Lin and S. Nadjm-Tehrani, "Timing patterns and correlations in spontaneous SCADA traffic for anomaly detection," in *Proc. 22nd Int. Symp. Res. Attacks, Intrusions Defenses (RAID)*, 2019, pp. 73–88.
[35] C.-Y. Lin and S. Nadjm-Tehrani, "Protocol study and anomaly detection for server-driven traffic in SCADA networks," *Int. J. Crit. Infrast. Protect.*, vol. 42, Sep. 2023, Art. no. 100612.
[36] M. Alanazi, A. Mahmood, and M. J. M. Chowdhury, "SCADA vulnerabilities and attacks: A review of the state-of-the-art and open issues," *Comput. Secur.*, vol. 125, Feb. 2023, Art. no. 103028.
[37] P. M. Laso, D. Brosset, and J. Puentes, "Analysis of quality measurements to categorize anomalies in sensor systems," in *Proc. Comput. Conf.*, 2017, pp. 1330–1338.
[38] "ICS Dataset for Smart-GRid Anomaly Detection," Dataset, dataport. Accessed: Aug. 9, 2023. [Online]. Available: https://ieee-dataport.org/documents/ics-dataset-smart-grid-anomaly-detection#files

uthorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on October 01,2024 at 17:05:14 UTC from IEEE Xplore. Restrictions apply